Q32

Can information flow through a model be quantified, for example with entropy?

Module author: KUKLA DLS 2026

Anchor lecture L2.1 (From Next-Token Prediction to Capabilities), Week 4. Source survey response #38.

1 The question, made precise

"Can we quantify the information flowing through a model?" is really three different questions wearing one coat. Separating them is most of the work:

1. **Output uncertainty.** How unsure is the model about the *next token*? This is the entropy of the predictive distribution $p(x_t \mid x_{<t})$. It is cheap, exact, and the single most useful number you can read off a language model.
2. **Compression / modelling quality.** How many *bits per token* does the model need to encode real text? This is cross-entropy, and it is exactly what pre-training minimises. "A language model is a compressor" (Delétang et al. 2023) is this statement taken literally.
3. **Internal information flow.** How much information does one layer, head, or token position carry about another, or about the input/label? This is the hard one. It uses mutual information between *activations*, and it is where most of the subtlety (and most of the bad plots) live.

The short answer to the survey question is **yes for (1) and (2) — exactly and cheaply — and "yes but carefully" for (3)**, because estimating mutual information between high-dimensional continuous activations is a genuinely hard statistical problem, and several famous "information flow" stories turned out to be artefacts of the estimator rather than facts about the network.

2 Entropy, cross-entropy, and perplexity (the exact part)

For a discrete next-token distribution p over vocabulary $\mathcal{V}$, the **Shannon entropy** [7] is

$$H(p) = -\sum_{v \in \mathcal{V}} p(v)\, \log_2 p(v) \quad \text{(bits)}.$$

$H = 0$ means the model is certain; $H = \log_2 |\mathcal{V}|$ means uniform guessing. This is a per-step read-out of "how much the model is hedging."

When we score the model against *real* text $x_{1:T}$, we use the **cross-entropy** of the data under the model,

$$\mathrm{CE} = -\frac{1}{T}\sum_{t=1}^{T} \log_2 p_\theta(x_t \mid x_{<t}),$$

and **perplexity** $\mathrm{PPL} = 2^{\mathrm{CE}}$ (or e^{CE} in nats). Cross-entropy decomposes as

$$\mathrm{CE} = \underbrace{H(p_{\text{data}})}_{\text{irreducible}} + \underbrace{D_{\mathrm{KL}}(p_{\text{data}} \,\|\, p_\theta)}_{\text{what training removes}},$$

which is why minimising cross-entropy *is* minimising the KL divergence from the model to the true distribution. The irreducible term is the entropy rate of language itself — Shannon's 1951

estimate [8] for English was roughly 1 bit per character; modern byte-level LMs get close to the same ballpark, which is a neat closing of a 70-year-old loop.

Compression is not a metaphor

A model with cross-entropy c bits/token induces, via *ideal* arithmetic coding, a code length that approaches c bits/token asymptotically — real encoders pay a small finite-message and implementation overhead, and c itself depends on the model and the corpus. Delétang et al. [2023] ran this the other way: they used a frozen LLM as the probability model inside a standard compressor and beat `gzip`/`PNG` on text *and* images. "Bits per token" measures modelling quality *relative to a given task and data distribution* — it is not a free-floating "quality" scalar, but on a fixed distribution lower bits/token is a strictly better model of that distribution.

3 Three things entropy buys you in practice

(a) A confidence / hallucination signal. Predictive entropy is a signal about the *model's output distribution*, not a ground-truth correctness detector. Low token entropy does *not* guarantee a correct answer (a model can be confidently wrong), and high entropy does *not* guarantee an error. What entropy *can* do, *after calibration and task-specific validation*, is support selective prediction and abstention: a high-entropy (or low logit-margin) decision-critical token is a useful flag that the model is uncertain and may warrant deferral [3]. These methods underlie practical abstention and selective-prediction (cf. L2.5, Q10–Q12). A subtlety students should internalise: entropy is computed *after* the temperature scaling and softmax, so it measures the *sampling* distribution, not some Platonic belief.

(b) Entropy-guided decoding. Temperature, top-k, top-p (nucleus), and min-p sampling are all entropy re-shaping operators. Nucleus sampling [3] exists precisely because the *tail* of a high-entropy distribution contains most of the degeneracy. "Entropy collapse" during RL fine-tuning — the policy becoming over-confident and losing exploration — is a live failure mode in RLHF/GRPO training (relevant to Q04/Q05/Q26).

(c) A training diagnostic. Plotting mean predictive entropy over training shows the model moving from near-uniform ($\log_2 |\mathcal{V}| \approx 15$–$17$ bits for a 50k vocab) down to a few bits as it learns the statistics of language. It is one of the cleanest "the model is learning something" curves you can draw.

4 Internal information flow (the careful part)

Now the ambitious reading: can we quantify information moving *between parts* of the network? The natural tool is **mutual information**

$$I(X;Y) = H(X) - H(X \mid Y) = \sum_{x,y} p(x,y) \log_2 \frac{p(x,y)}{p(x)p(y)},$$

applied to, say, a hidden representation $Z = f_\ell(X)$ at layer ℓ versus the input X or the label Y. This is the setting of the **Information Bottleneck** view of deep learning [9, 10], which claimed networks undergo a "compression phase" where $I(Z;X)$ drops while $I(Z;Y)$ stays high.

The cautionary tale every student should hear

Saxe et al. [2018] showed the famous "compression phase" is largely an *artefact of binning a continuous variable to estimate entropy*: with a double-sided saturating nonlinearity (`tanh`) you see compression; with `ReLU` you often don't. The deterministic-mapping subtlety matters and splits by variable type: for a *deterministic* network f with *continuous* X, the standard differential-entropy $I(Z;X)$ is ill-defined/infinite (an invertible-in-practice map preserves all information), whereas for *discrete* X it is finite and bounded by $H(X)$. Either way, the number you actually *plot* is a property of your discretisation/estimator. **Mutual information between activations is not a free lunch; the estimator *is* the measurement.**

This does not mean internal information is unmeasurable — it means you must be honest about what you are estimating:

- **Discrete-by-construction quantities are safe.** Attention weights are a probability distribution per query, so their entropy is exact and interpretable (a low-entropy head is "pointing"; a high-entropy head is "averaging"). This is a legitimate, assumption-free information read-out.
- **MI needs a principled estimator.** KSG [4] for low dimensions; MINE / InfoNCE [1, 11] for high dimensions — and even these are *bounds*, often loose, and notoriously hard above a few nats [5].
- **Perturbation beats estimation for causal flow.** If the real question is "does information from token i reach position j?", ablating/patching the path and measuring the change in output (activation patching, causal tracing; cf. Q14/Q27) answers the *causal* question directly, without estimating a high-dimensional MI at all.

5 What to tell a student who asks this question

1. **Yes**, and you probably want *predictive entropy* and *cross-entropy/perplexity* — they are exact, cheap, and tell you most of what you want (uncertainty, compression quality, training progress, decoding behaviour).
2. For *internal* flow, attention entropy is a safe exact read-out; layer↔input/label mutual information is seductive but estimator-dependent — treat any "information plane" plot as a claim about the estimator until proven otherwise.
3. When you want *causal* flow, don't estimate information — intervene (patch/ablate) and measure the output change.

The accompanying code (`code/entropy_flow.py`) computes all of the "safe" quantities — per-token predictive entropy, corpus cross-entropy/perplexity, per-head attention entropy, and the H+KL decomposition — on *synthetic* next-token and attention distributions, so the file is dependency-free (numpy only), deterministic, and needs no model download. The arithmetic is identical to what you would run on a real model: swap the synthetic `softmax(logits)` for your model's actual output logits and attention tensors and nothing in the entropy/MI math changes. It also includes the deliberate cautionary demo — an information-plane estimate that visibly changes when you only change the number of bins, so students *see* the artefact rather than just read about it. Running the same pipeline on a real small model (e.g. reading logits and attention from a local transformer) is listed as a project in `projects.tex`, not claimed as this module's execution evidence.

References

[1] Mohamed Ishmael Belghazi, Aristide Baratin, Sai Rajeswar, Sherjil Ozair, Yoshua Bengio, Aaron Courville, and R. Devon Hjelm. Mutual information neural estimation. In *International Conference on Machine Learning (ICML)*, 2018. arXiv:1801.04062.

[2] Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Mattern, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Orseau, Marcus Hutter, and Joel Veness. Language modeling is compression. *arXiv preprint arXiv:2309.10668*, 2023.

[3] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1904.09751.

[4] Alexander Kraskov, Harald Stögbauer, and Peter Grassberger. Estimating mutual information. *Physical Review E*, 69(6):066138, 2004.

[5] David McAllester and Karl Stratos. Formal limitations on the measurement of mutual information. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2020. arXiv:1811.04251.

[6] Andrew M. Saxe, Yamini Bansal, Joel Dapello, Madhu Advani, Artemy Kolchinsky, Brendan D. Tracey, and David D. Cox. On the information bottleneck theory of deep learning. In *International Conference on Learning Representations (ICLR)*, 2018.

[7] C. E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.

[8] C. E. Shannon. Prediction and entropy of printed English. *The Bell System Technical Journal*, 30(1):50–64, 1951.

[9] Ravid Shwartz-Ziv and Naftali Tishby. Opening the black box of deep neural networks via information. *arXiv preprint arXiv:1703.00810*, 2017.

[10] Naftali Tishby and Noga Zaslavsky. Deep learning and the information bottleneck principle. In *IEEE Information Theory Workshop (ITW)*, 2015. arXiv:1503.02406.

[11] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. In *arXiv preprint arXiv:1807.03748*, 2018.